



FICHIERS À CRÉER/REEMPLACER

LOCALEMENT

lib/utils/manufacturerResolver.js

```
// lib/utils/manufacturerResolver.js
// Résolution d'identité des fabricants selon la Rule 24 (normalisation in
const crypto = require('crypto');

class ManufacturerResolver {
  /**
  <ul>
  <li>Normalise un nom de fabricant selon la Rule 24</li>
  <li>@param {string} name - Nom brut du fabricant</li>
  <li>@returns {string|null} Nom normalisé ou null si invalide</li>
  </ul>
  */
  static normalize(name) {
    if (!name || typeof name !== 'string') return null;

    return name
      .trim() // Supprime les espaces d
      .normalize('NFKD') // Décompose les caractères
      .replace(/[\u0300-\u036f]/g, '') // Supprime les accents/di
      .toLowerCase() // Insensible à la casse
      .replace(/^[a-z0-9]+/g, '_') // Remplace les non-alphan
      .replace(/^[^_]+_|_+$/g, '') // Supprime les _ en début
      .replace(/_+/g, '_'); // Collapse les underscore
  }

  /**
  <ul>
  <li>Résout un nom brut vers une forme canonique via un mapping</li>
  <li>@param {string} rawName - Nom brut</li>
  <li>@param {Object} mapping - Table de mapping variants → canonique</li>
  <li>@returns {string|null} Nom canonique ou null si inconnu</li>
  </ul>
  */
}
```

```

</ul>
    */
    static resolve(rawName, mapping = {}) {
        const normalized = this.normalize(rawName);
        if (!normalized) return null;

        // Recherche dans le mapping fourni
        if (mapping[normalized]) {
            return mapping[normalized];
        }

        // Fallback : retourne la forme normalisée comme canonique
        return normalized;
    }

    /**
<ul>
<li>Génère un ID unique stable pour un fabricant</li>
<li>@param {string} canonicalName - Nom canonique</li>
<li>@returns {string|null} Hash SHA-256 tronqué (12 caractères)</li>
</ul>
    */
    static generateId(canonicalName) {
        if (!canonicalName) return null;
        return crypto
            .createHash('sha256')
            .update(canonicalName)
            .digest('hex')
            .slice(0, 12);
    }

    /**
<ul>
<li>Construit une table de mapping à partir d'une liste d'entrées</li>
<li>@param {Array<{variants: string[], canonical: string}>} entries</li>
<li>@returns {Object} Mapping normalized → canonical</li>
</ul>
    */
    static buildMapping(entries) {
        const mapping = {};

```

```

    for (const entry of entries) {
        const canonical = this.normalize(entry.canonical);
        if (!canonical) continue;

        // Mappe tous les variants vers le canonique
        for (const variant of entry.variants) {
            const normalized = this.normalize(variant);
            if (normalized) {
                mapping[normalized] = canonical;
            }
        }
    }

    // Assure que le canonique pointe vers lui-même
    mapping[canonical] = canonical;
}

return mapping;
}
}

module.exports = ManufacturerResolver;

```

lib/drivers/DynamicDriverMatcher.js (Extrait mis à jour)

```

// lib/drivers/DynamicDriverMatcher.js
const ManufacturerResolver = require('../utils/manufacturerResolver');

class DynamicDriverMatcher {
    constructor() {
        this.fingerprintIndex = new Map(); // Index O(1) pour le lookup rapide
        this.manufacturerMapping = this._loadManufacturerMapping();
    }

    /**
     <ul>

```

- Trouve un driver correspondant à un device découvert
- @param {Object} deviceInfo - Informations du device Zigbee
- @returns {Driver|null} Driver correspondant ou null

```
    */
    findDriver(deviceInfo) {
        // 1. Normaliser le manufacturerName (Rule 24)
        const rawManufacturer = deviceInfo.manufacturerName;
        const normalizedManufacturer = ManufacturerResolver.normalize(rawManuf
        const canonicalManufacturer = ManufacturerResolver.resolve(
            rawManufacturer,
            this.manufacturerMapping
        );

        // 2. Normaliser le modelId
        const rawModelId = deviceInfo.modelId;
        const normalizedModelId = this._normalizeDeviceId(rawModelId);

        // 3. Recherche dans l'index avec combinaisons possibles
        const candidates = [
            `canonicalManufacturer:{normalizedModelId}`,
            `normalizedManufacturer:{normalizedModelId}`,
            `*:${normalizedModelId}`, // Wildcard manufacturer
            `${canonicalManufacturer}:*`, // Wildcard modelId
        ];

        for (const key of candidates) {
            const driver = this.fingerprintIndex.get(key);
            if (driver) {
                this.log('Driver matched', {
                    manufacturer: rawManufacturer,
                    modelId: rawModelId,
                    matchedKey: key
                });
                return driver;
            }
        }

        // 4. Fallback : recherche floue si aucun match exact
        return this._fuzzyMatch(deviceInfo);
    }
}
```

```

    }

    /**
    <ul>
    <li>Normalise un device ID (insensible à la casse, formats multiples)</li>
    <li>@private</li>
    </ul>
    */
    _normalizeDeviceId(deviceId) {
        if (!deviceId) return null;
        return deviceId
            .trim()
            .toLowerCase()
            .replace(/[-_\\s]+/g, '') // Supprime séparateurs
            .replace(/^0x/i, '');    // Supprime préfixe hex
    }

    /**
    <ul>
    <li>Charge le mapping des fabricants depuis data/manufacturers.json</li>
    <li>@private</li>
    </ul>
    */
    _loadManufacturerMapping() {
        try {
            const manufacturersData = require('../../data/manufacturers.json');
            return ManufacturerResolver.buildMapping(manufacturersData.mapping_r
        } catch (error) {
            this.log('Failed to load manufacturer mapping', error.message);
            return {};
        }
    }

    /**
    <ul>
    <li>Construit l'index de fingerprints pour une recherche O(1)</li>
    <li>@param {Array} drivers - Liste des drivers avec leurs fingerprints</li>
    </ul>
    */
    buildIndex(drivers) {

```

```

    for (const driver of drivers) {
      // Index par combinaison manufacturer:modelId normalisés
      for (const fp of driver.fingerprints || []) {
        const key = `fp.manufacturer:${fp.modelId}`.toLowerCase();
        this.fingerprintIndex.set(key, driver);

        // Index alternatif avec wildcards pour matching flexible
        this.fingerprintIndex.set(`*:${fp.modelId}`, driver);
        this.fingerprintIndex.set(`${fp.manufacturer}:*`, driver);
      }
    }
  }

  /**
<ul>
<li>Recherche floue (fallback) si aucun match exact n'est trouvé</li>
<li>@private</li>
</ul>
  */
  _fuzzyMatch(deviceInfo) {
    // Implémentation de recherche floue basée sur la distance de Levenshtein
    // Cette méthode doit être implémentée en fonction des besoins spécifiques
    this.log('_fuzzyMatch not implemented yet', deviceInfo);
    return null;
  }
}

module.exports = DynamicDriverMatcher;

```

.eslintrc.json

```

{
  "env": {
    "node": true,
    "es2022": true,
    "homey": true
  },
  "parserOptions": {

```

```

    "ecmaVersion": 2022,
    "sourceType": "module"
  },
  "extends": ["eslint:recommended"],
  "rules": {
    " |===== |",
    " | PRÉVENTION SYNTAXE CRITIQUE (Bloquants v7.4.6/v7.4.8) |",
    " |===== |",

    "no-unmatched-parens": "error",
    "no-unexpected-multiline": "error",
    "no-conditional-assignment": "error",
    "no-unused-expressions": "error",
    "no-self-compare": "error",
    "no-extra-parens": ["error", "all", { "nestedBinaryExpressions": false }],
    "no-mixed-operators": "warn",

    " |===== |",
    " | QUALITÉ CODE & MAINTENABILITÉ |",
    " |===== |",

    "curly": ["error", "all"],
    "eqeqeq": ["error", "smart"],
    "no-var": "error",
    "prefer-const": "warn",
    "prefer-template": "warn",
    "no-console": "off",
    "no-debugger": "error",
    "complexity": ["warn", 15],
    "max-depth": ["warn", 4],
    "max-nested-callbacks": ["warn", 3],
    "max-params": ["warn", 5],
    "max-statements": ["warn", 50],
    "max-lines": ["warn", 300],

    " |===== |",
    " | SPÉCIFIQUES HOMEY SDK3 & SÉCURITÉ |",
    " |===== |",

    "no-return-await": "warn",
    "handle-callback-err": ["error", "^(err|error)$"],
    "callback-return": ["warn", ["callback", "cb", "next"]],
    "no-eval": "error",
  }
}

```

```

    "no-implied-eval": "error",
    "no-new-func": "error",
    "no-script-url": "error",
    "no-unsafe-negation": "error",
    "no-throw-literal": "error"
  },
  "globals": {
    "Homey": "readonly",
    "module": "writable",
    "require": "writable",
    "console": "writable",
    "Buffer": "readonly",
    "setTimeout": "readonly",
    "setInterval": "readonly",
    "clearTimeout": "readonly",
    "clearInterval": "readonly"
  },
  "ignorePatterns": [
    "node_modules/",
    ".homey/",
    "build/",
    "drivers/",
    "data/",
    "tmp/",
    ".cache/",
    "coverage/"
  ]
}

```

scripts/refactor-hybrid-drivers.js

```

// scripts/refactor-hybrid-drivers.js
// Script pour supprimer le suffixe `_hybrid` des dossiers de drivers et m

const fs = require('fs');
const path = require('path');

console.log('🔄 Refactoring drivers: suppression du suffixe _hybrid...');

```



```

        match.replace(/_hybrid([\\""])/, '$1'));

    if (content !== updated) {
        console.log(`✎ Mise à jour des imports dans ${file}`);
        fs.writeFileSync(filePath, updated);
    }
});
}

// 3. Mettre à jour app.json si références aux drivers _hybrid
const appJsonPath = path.join(__dirname, '..', 'app.json');
if (fs.existsSync(appJsonPath)) {
    const app = JSON.parse(fs.readFileSync(appJsonPath, 'utf8'));
    let changed = false;

    if (app.drivers) {
        for (const [id, driver] of Object.entries(app.drivers)) {
            if (id.endsWith('_hybrid')) {
                const newId = id.replace(/_hybrid$/, '');
                app.drivers[newId] = driver;
                delete app.drivers[id];
                changed = true;
                console.log(`↺ Mise à jour de l'ID du driver: id→{newId}`);
            }
        }
    }

    if (changed) {
        fs.writeFileSync(appJsonPath, JSON.stringify(app, null, 2) + '\n');
        console.log('✅ app.json mis à jour');
    }
}

console.log('✅ Refactorisation _hybrid terminée');
console.log('📋 Prochaines étapes: exécuter validate-all.sh et pousser sur

```

test/critical/manufacturerResolver.test.js

```
// test/critical/manufacturerResolver.test.js
import { describe, it, assert } from 'node:test';
import ManufacturerResolver from '../lib/utils/manufacturerResolver.js';

describe('ManufacturerResolver.normalize (Rule 24)', () => {
  const testCases = [
    { input: 'Tuya', expected: 'tuya' },
    { input: 'tuya', expected: 'tuya' },
    { input: 'TUYA', expected: 'tuya' },
    { input: 'Tuya_Smart', expected: 'tuya_smart' },
    { input: 'tuya-inc', expected: 'tuya_inc' },
    { input: ' Tuya ', expected: 'tuya' },
    { input: 'Tuya\u00A0Smart', expected: 'tuya_smart' }, // espace insécable
    { input: null, expected: null },
    { input: '', expected: null },
    { input: 'Tüyä', expected: 'tuya' }, // avec accents
  ];

  for (const { input, expected } of testCases) {
    it(`normalise "input" en "expected"`, () => {
      assert.strictEqual(ManufacturerResolver.normalize(input), expected);
    });
  }
});

describe('ManufacturerResolver.resolve with mapping', () => {
  const mapping = {
    'tuya_smart': 'tuya',
    'tuyainc': 'tuya',
    'lidl': 'lidl_silvercrest'
  };

  it('résout les variants connus vers leur forme canonique', () => {
    assert.strictEqual(
      ManufacturerResolver.resolve('Tuya_Smart', mapping),
      'tuya'
    );
    assert.strictEqual(

```

```

        ManufacturerResolver.resolve('LIDL', mapping),
        'lidl_silvercrest'
    );
});

it('retourne la forme normalisée comme canonique pour les variants inconnus', () => {
    assert.strictEqual(
        ManufacturerResolver.resolve('UnknownBrand', mapping),
        'unknownbrand'
    );
});
});

```

data/manufacturers.json

```

{
  "mapping_rules": [
    {
      "canonical": "tuya",
      "variants": [
        "Tuya", "tuya", "TUYA", "Tuya_Smart", "TuyaInc", "tuya-inc",
        "Tuya\u00A0Smart", " TUYA ", "tuya.smart", "TUYA_SMART"
      ],
      "device_id_patterns": ["^TS\\d{4}$", "^zigbee_<em>", "^tuya_</em>"],
      "metadata": {
        "country": "CN",
        "protocols": ["zigbee", "wifi", "matter"],
        "first_seen": "2020-01-01"
      }
    },
    {
      "canonical": "lidl_silvercrest",
      "variants": [
        "Lidl", "SilverCrest", "silvercrest", "LIDL_SILVERCREST",
        "Silvercrest", "silver-crest", "LIDL"
      ],
      "device_id_patterns": ["^TS011F$", "^lidl_.*"],
      "metadata": {

```

```

        "country": "DE",
        "protocols": ["zigbee"],
        "first_seen": "2021-11-01"
    }
},
"generated_mapping": {
    "tuya": "tuya",
    "tuya_smart": "tuya",
    "tuyainc": "tuya",
    "lidl": "lidl_silvercrest",
    "silvercrest": "lidl_silvercrest"
}
}

```



ÉTAPES D'EXÉCUTION & PUSH

Étapes Locales (À Exécuter dans votre terminal)

1. Cloner le dépôt (si ce n'est pas déjà fait)

```
git clone https://github.com/dlnraja/com.tuya.zigbee.git
cd com.tuya.zigbee
```

2. Installer les dépendances de développement

```
npm ci
```

3. Créer les dossiers requis

```
mkdir -p lib/utils lib/drivers data scripts test/critical .github/workflows
```

4. Copier les fichiers générés localement

- Copiez le contenu de chaque bloc de code ci-dessus dans son fichier

- Par exemple, copiez le contenu de `lib/utils/manufacturerResolver.js`

5. Appliquer le refactor des drivers _hybrid

```
node scripts/refactor-hybrid-drivers.js
```

6. Valider toutes les modifications locales

```
bash scripts/validate-all.sh
```

```
# 7. Committer les changements
```

```
git add .
```

```
git commit -m "🐛 Correctifs critiques + refactor permissif + Rule 24\n\n-
```

```
# 8. Pousser sur GitHub
```

```
git push origin master
```

Déclenchement Automatique des Workflows GitHub Actions ("Shadow Mode")

Une fois les changements poussés sur la branche `master`, les workflows suivants se déclencheront automatiquement en arrière-plan : - `syntax-check.yml` : Vérifie la syntaxe de tous les fichiers `.js` dans `lib/`. - `refactor-validation.yml` : Vérifie qu'il ne reste plus aucun dossier `*_hybrid` dans `drivers/`. - `dependency-audit.yml` : Effectue un audit de sécurité npm quotidien. - `gmail-diagnostics-anonymize.yml` : Récupère et anonymise les rapports de crash provenant de Gmail.

Aucune action manuelle n'est requise après le `git push`. Les workflows CI/CD s'exécutent silencieusement et génèrent des artefacts (rapports, builds) qui sont stockés en tant qu'artefacts GitHub. Aucun message public n'est publié.



EXPLICATION DU MODE SHADOW CI/CD

Le mode "Shadow" signifie que **les workflows GitHub Actions fonctionnent entièrement en arrière-plan sans aucune interaction publique**. Voici comment cela fonctionne :

1. **Déclenchement Silencieux** : Lorsque vous poussez vos modifications locales sur la branche `master`, les workflows définis dans `.github/workflows/*.yml` se déclenchent automatiquement `[[unique_id]]`.
2. **Validation Automatique** : Chaque workflow exécute sa tâche spécifique (vérification de syntaxe, audit de sécurité, récupération d'emails) sans envoyer de notification à personne.

3. **Stockage des Artefacts** : Les résultats de ces workflows (rapports de diagnostic, builds, logs) sont stockés comme "artefacts GitHub". Ils sont accessibles uniquement aux personnes ayant accès au dépôt GitHub, mais ne sont jamais publiés sur le forum Homey ni partagés publiquement `[[unique_id]]`.
4. **Absence Totale d'Annonces Publiques** : Contrairement à un flux de travail classique, aucun message n'est posté sur le forum Homey, aucun email n'est envoyé à la communauté, et aucun tag de version n'est créé sur GitHub. Tout est strictement interne `[[unique_id]]`.
5. **Préparation pour le Développeur** : Une fois les workflows terminés avec succès, ils produisent des artefacts prêts à l'emploi (par exemple, un build valide dans `build/`). Le développeur peut alors décider de publier cette version sur le canal test Homey, mais c'est une action manuelle et distincte `[[unique_id]]`.

En résumé, le mode "Shadow" transforme GitHub Actions en un système de validation et de préparation automatique, totalement invisible pour les utilisateurs finaux, garantissant que seules des versions rigoureusement testées et validées peuvent être considérées pour une publication éventuelle.



CHECKLIST FINALE

- ☐ ☒ Les fichiers `manufacturerResolver.js`, `DynamicDriverMatcher.js`, `.eslintrc.json`, `refactor-hybrid-drivers.js`, `manufacturers.json`, et `manufacturerResolver.test.js` ont été créés localement avec le code fourni.
- ☐ ☒ Le script `refactor-hybrid-drivers.js` a été exécuté avec succès.
- ☐ ☒ La commande `bash scripts/validate-all.sh` passe sans erreur.
- ☐ ☒ Un commit local a été effectué avec un message détaillé incluant les références aux correctifs et aux améliorations.
- ☐ ☒ La commande `git push origin master` a été exécutée pour pousser les changements sur GitHub.
- ☐ ☒ Les workflows GitHub Actions (`syntax-check.yml`, `refactor-validation.yml`, etc.) sont configurés dans le dépôt et vont maintenant s'exécuter automatiquement en arrière-plan.
- ☐ ☒ Il n'y a aucune intention de faire une annonce publique sur le forum Homey avant la validation complète des workflows CI/CD et la décision finale du développeur.